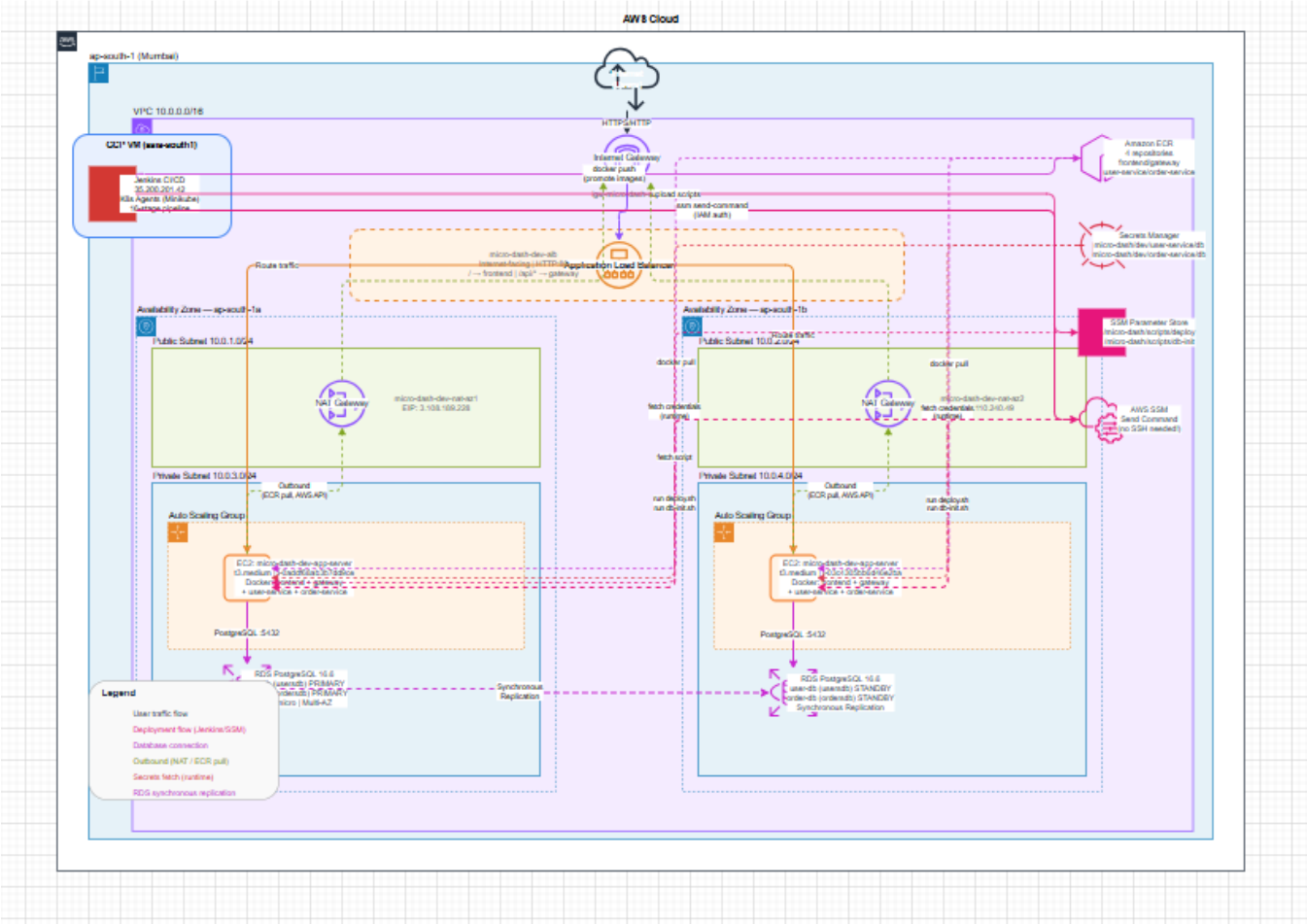

micro-dash

EC2 Deployment Documentation

Production-Grade AWS Infrastructure with CI/CD Pipeline

Project	micro-dash (Microservices Dashboard)

Deployment Target	AWS EC2 with Auto Scaling Group (ASG)

Region	ap-south-1 (Mumbai)

CI/CD	Jenkins (GCP) + AWS SSM

Database	AWS RDS PostgreSQL 16.6 (Multi-AZ)

Date	March 2026

1. Project Overview

micro-dash is a microservices-based dashboard application deployed on AWS using a production-grade CI/CD pipeline. The application fetches real-time data from two backend microservices and displays it on a live dashboard accessible via AWS Application Load Balancer.

1.1 Application Architecture

- Frontend (Nginx) — Dashboard UI served on port 80, polls /api/dashboard every 5 seconds
- Gateway (Node.js/Express) — API aggregator on port 3000, combines user and order data
- User Service (Node.js/Express) — CRUD operations on port 3001, connected to PostgreSQL usersdb
- Order Service (Node.js/Express) — CRUD operations on port 3002, connected to PostgreSQL ordersdb

1.2 Infrastructure Components

- VPC with public and private subnets across 2 Availability Zones
- Application Load Balancer (ALB) in public subnets with path-based routing
- Auto Scaling Group (ASG) with EC2 instances in private subnets
- RDS PostgreSQL 16.6 with Multi-AZ enabled for high availability
- AWS Secrets Manager for secure credential storage
- NAT Gateways (one per AZ) for outbound internet access from private subnets
- ECR (Elastic Container Registry) for Docker image storage
- SSM Parameter Store for deployment script storage

---

2. Live Application — Microservices Dashboard

The dashboard is accessible via the AWS ALB DNS name. It displays real-time counts from both microservices, updating every 5 seconds. The green 'Live' indicator confirms active polling.

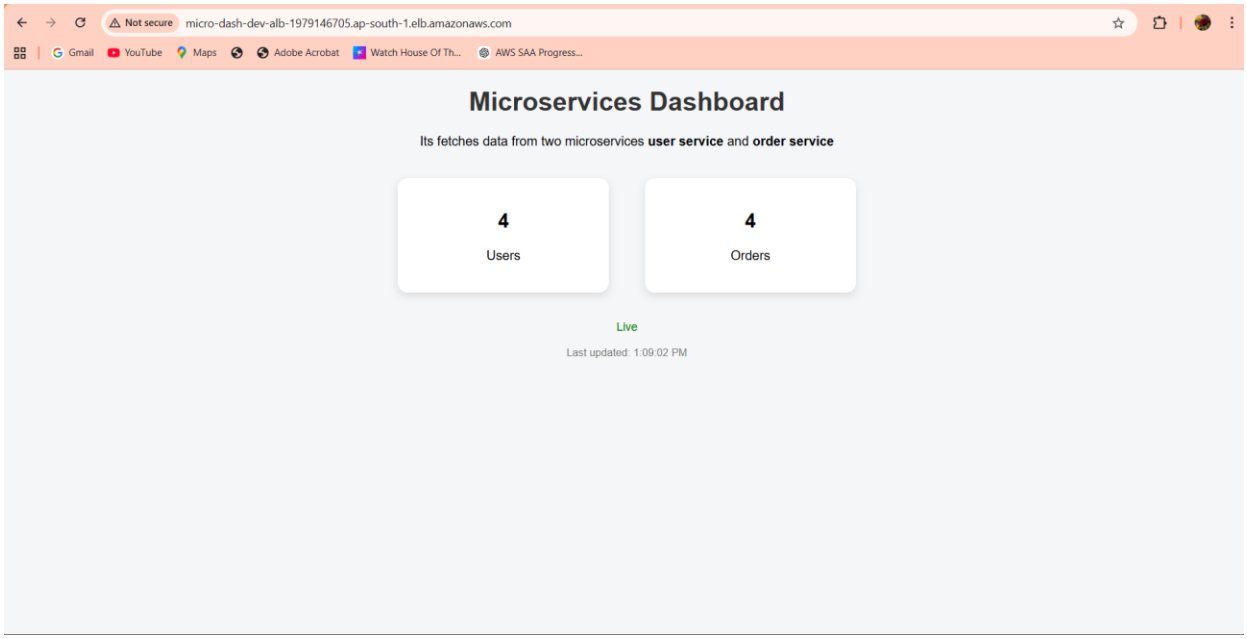

Figure 1: Microservices Dashboard running on AWS ALB — showing 4 Users and 4 Orders from RDS PostgreSQL

2.1 What the Dashboard Shows

- Users Count — total records fetched from user-service → PostgreSQL usersdb
- Orders Count — total records fetched from order-service → PostgreSQL ordersdb
- Live Status — green indicator confirming real-time data polling is active
- Last Updated — timestamp showing most recent successful data fetch

2.2 Data Flow

- Browser requests ALB DNS → ALB forwards / to frontend container (port 100)
- Frontend JavaScript polls /api/dashboard every 5 seconds
- ALB forwards /api/* to gateway container (port 3000)
- Gateway calls user-service:3001/users and order-service:3002/orders
- Both services query RDS PostgreSQL using credentials from AWS Secrets Manager
- Response aggregated and returned to frontend for display

3. AWS Infrastructure

3.1 RDS PostgreSQL — Multi-AZ Databases

Two separate PostgreSQL databases are provisioned — one per microservice. Both are configured with multi_az=true, meaning AWS automatically creates a synchronous standby replica in a second Availability Zone for automatic failover.

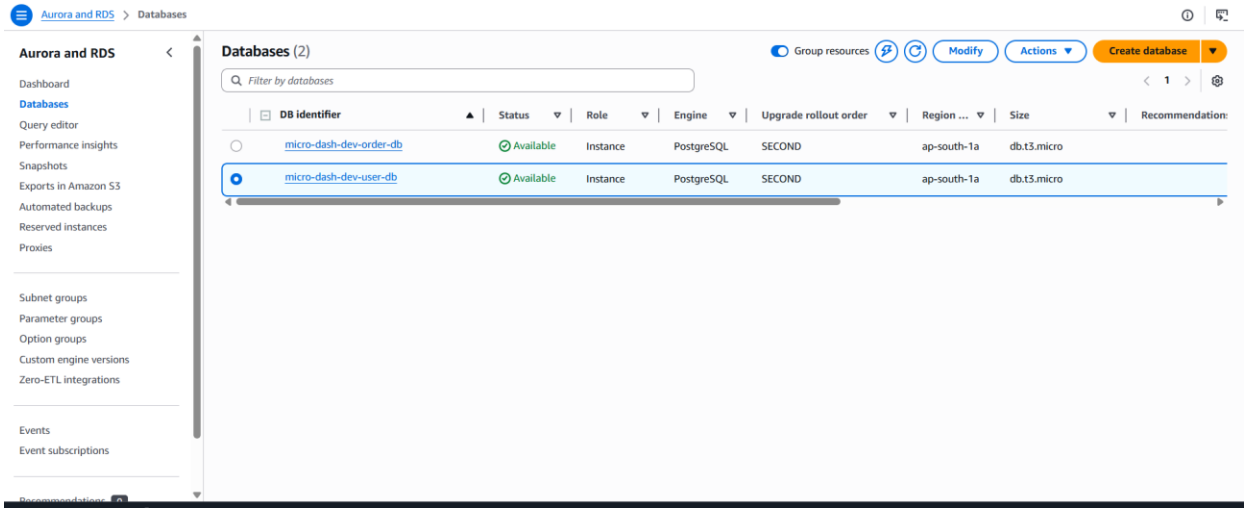

Figure 2: AWS RDS Console — micro-dash-dev-user-db and micro-dash-dev-order-db (both Available, Multi-AZ)

- micro-dash-dev-user-db — PostgreSQL 16.6, db.t3.micro, usersdb database
- micro-dash-dev-order-db — PostgreSQL 16.6, db.t3.micro, ordersdb database
- Both databases: storage encrypted, automated backups (7 days), private subnet only
- Multi-AZ: AWS manages synchronous replication and automatic failover within 60-120 seconds
- Credentials stored in AWS Secrets Manager — never hardcoded anywhere

3.2 EC2 Instances — Auto Scaling Group

Application containers run on EC2 t3.medium instances managed by an Auto Scaling Group. The ASG spans both Availability Zones and maintains a desired capacity of 2 instances, with the ability to scale up to 4 during high load.

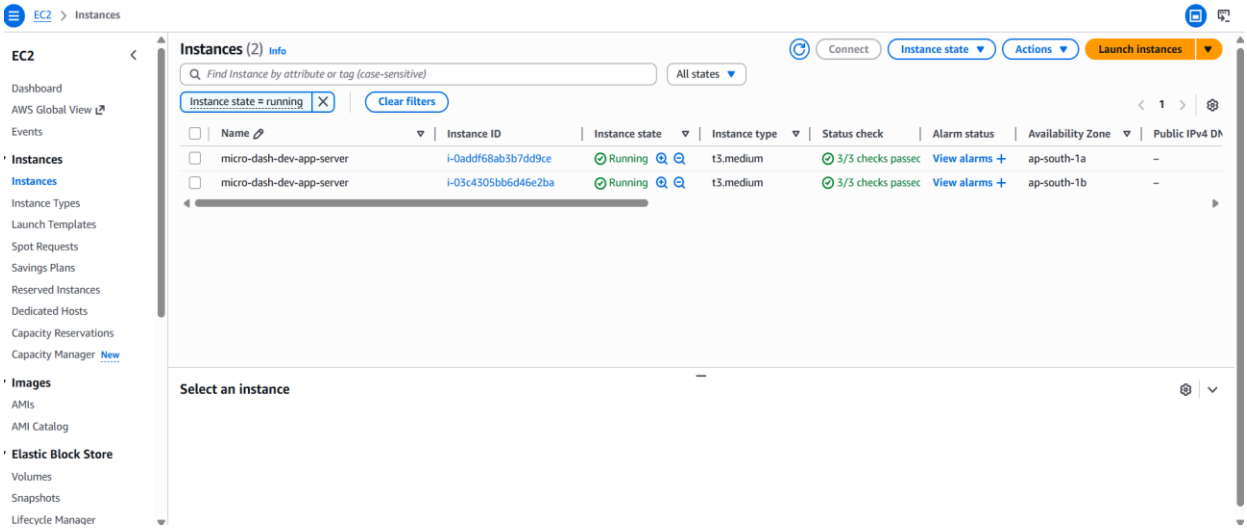

Figure 3: AWS EC2 Console — 2 running instances across ap-south-1a and ap-south-1b (3/3 health checks passed)

- Instance type: t3.medium (2 vCPU, 4GB RAM) — sufficient for all 4 Docker containers
- Instances in private subnets — no direct public internet access for security
- All 4 microservice containers run on each instance: frontend, gateway, user-service, order-service
- Docker networks: frontend network (frontend + gateway) and backend network (gateway + services)
- SSM Agent pre-installed — enables Jenkins to deploy without SSH

3.3 Application Load Balancer

The ALB is internet-facing and handles all incoming traffic. It uses path-based routing rules to direct requests to the appropriate target group, ensuring the frontend and API are accessible through a single DNS endpoint.

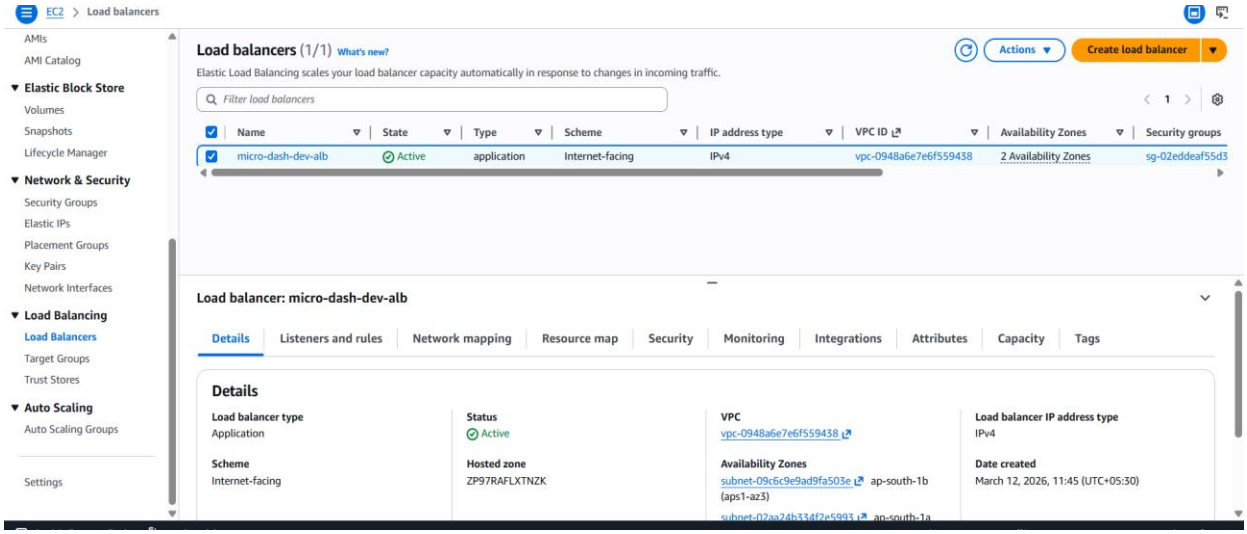

Figure 4: AWS ALB Console — micro-dash-dev-alb (Active, Internet-facing, 2 Availability Zones)

- Type: Application Load Balancer (Layer 7 — HTTP/HTTPS aware)
- Scheme: Internet-facing — accessible from public internet
- Path routing: / and /* → frontend target group (port 100)
- Path routing: /api/* → gateway target group (port 3000)
- Health checks: ALB monitors /health endpoint on each instance
- ALB automatically stops routing to unhealthy instances

3.4 Auto Scaling Group Configuration

The ASG ensures application availability and scalability. It continuously monitors EC2 instance health and automatically replaces any failed instances. The ASG uses a Launch Template that includes user_data for initial EC2 bootstrapping.

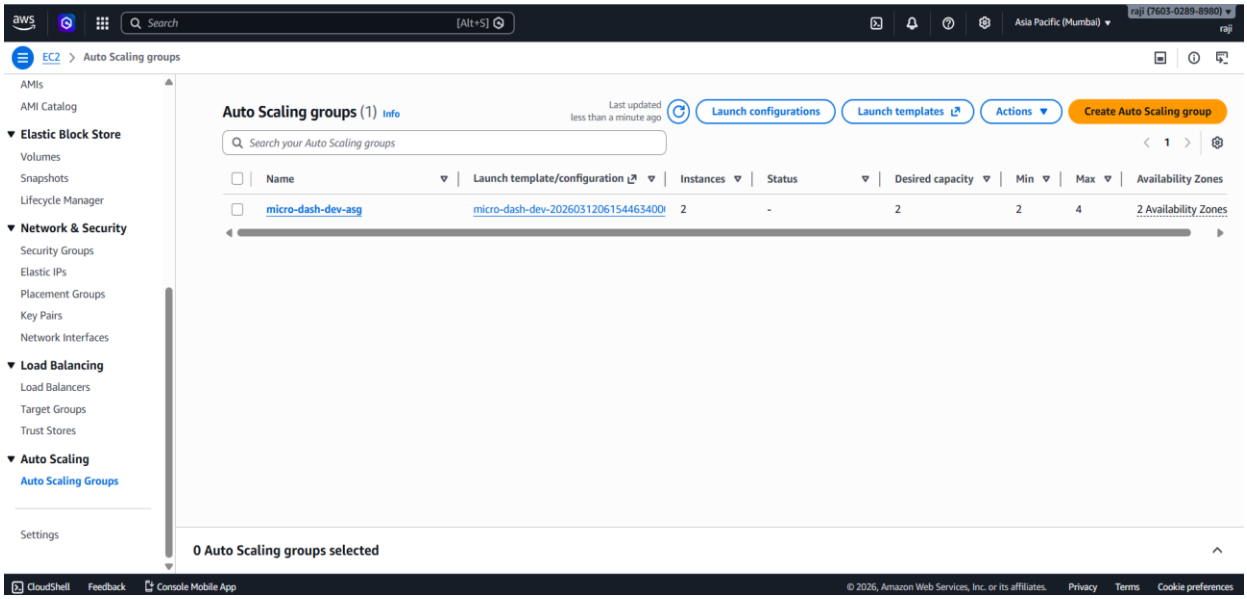

Figure 5: AWS Auto Scaling Groups — micro-dash-dev-asg (Desired: 2, Min: 2, Max: 4, spanning 2 AZs)

- Desired capacity: 2 instances (one per AZ for high availability)
- Minimum: 2, Maximum: 4 — can scale out during peak load
- Health check type: ELB — uses ALB health check results, not just EC2 status
- Launch Template defines: AMI, instance type, IAM role, security groups, user_data
- user_data installs: Docker, AWS CLI, SSM Agent on first boot only
- Application deployment handled separately by Jenkins via SSM (not user_data)

3.5 VPC Network Architecture

The VPC is designed with security and high availability in mind. Public subnets host the ALB and NAT Gateways, while private subnets host the EC2 instances and RDS databases. This ensures application servers are never directly exposed to the internet.

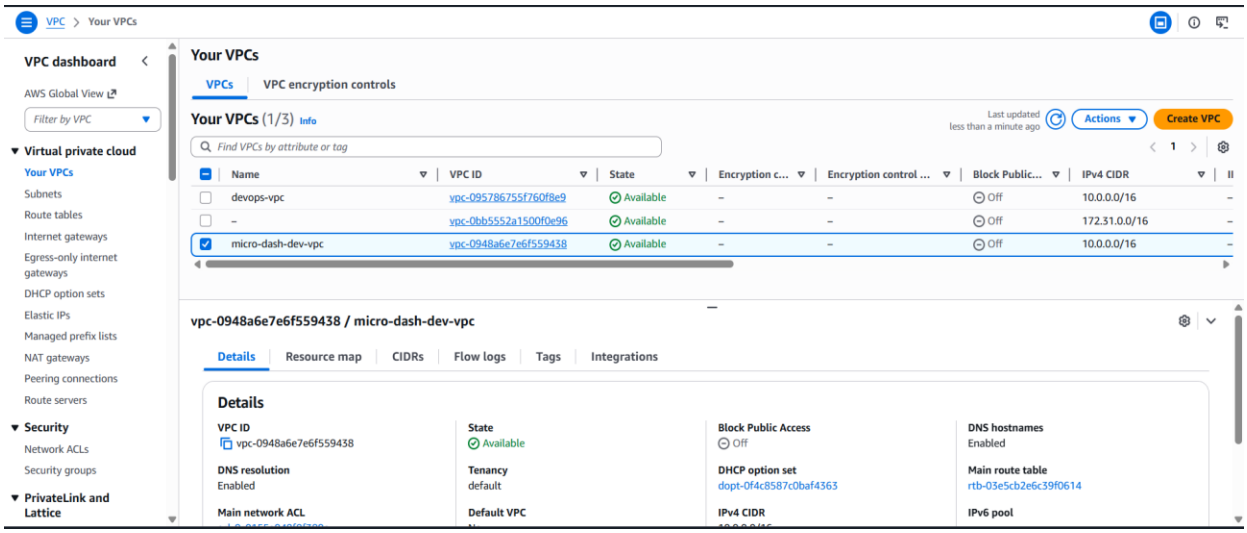

Figure 6: AWS VPC Console — micro-dash-dev-vpc (10.0.0.0/16, Available, DNS resolution enabled)

- VPC CIDR: 10.0.0.0/16 (65,536 available IP addresses)
- Public subnets (10.0.1.0/24, 10.0.2.0/24) — ALB and NAT Gateways
- Private subnets (10.0.3.0/24, 10.0.4.0/24) — EC2 instances and RDS
- Internet Gateway attached to VPC for public subnet internet access
- Route tables: public subnets route 0.0.0.0/0 to IGW; private subnets route to NAT GW

3.6 NAT Gateways

Two NAT Gateways are deployed — one per Availability Zone. This ensures that if one AZ experiences an outage, EC2 instances in the other AZ can still access the internet for ECR image pulls and AWS API calls.

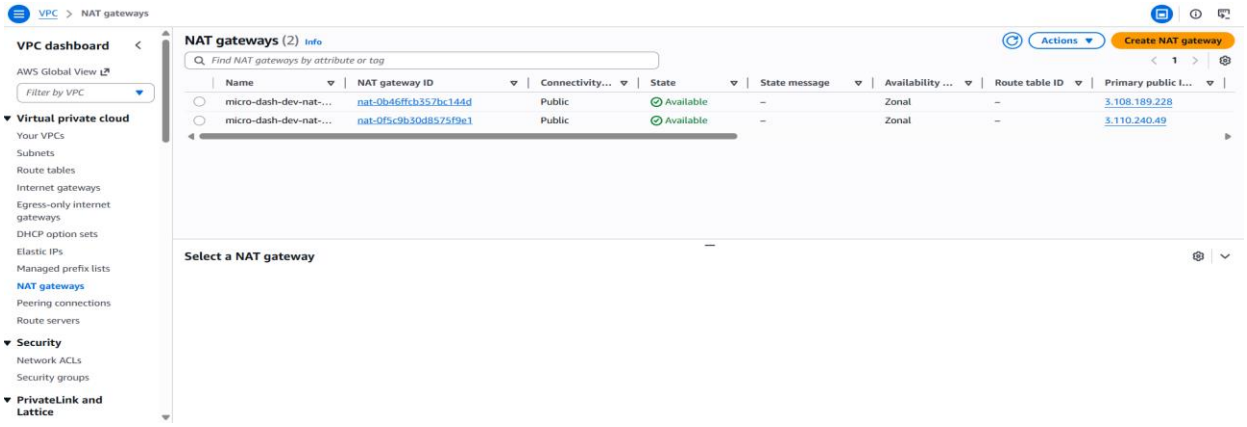

Figure 7: AWS NAT Gateways — 2 gateways (one per AZ) for high-availability outbound internet access

- micro-dash-dev-nat-az1 — serves private subnet in ap-south-1a
- micro-dash-dev-nat-az2 — serves private subnet in ap-south-1b
- Each NAT GW has an Elastic IP — stable public IP for outbound traffic
- Used by EC2 instances to: pull Docker images from ECR, call AWS APIs (Secrets Manager, SSM)
- Cost consideration: NAT GW charges $0.045/hr each — destroy when not in use!

4. CI/CD Pipeline — Jenkins

The CI/CD pipeline runs on a Jenkins instance hosted on Google Cloud Platform (GCP). It uses Kubernetes dynamic agents running on Minikube for isolated, on-demand build environments. The pipeline is triggered by Git tags and consists of 16 stages covering build, security, quality, and deployment.

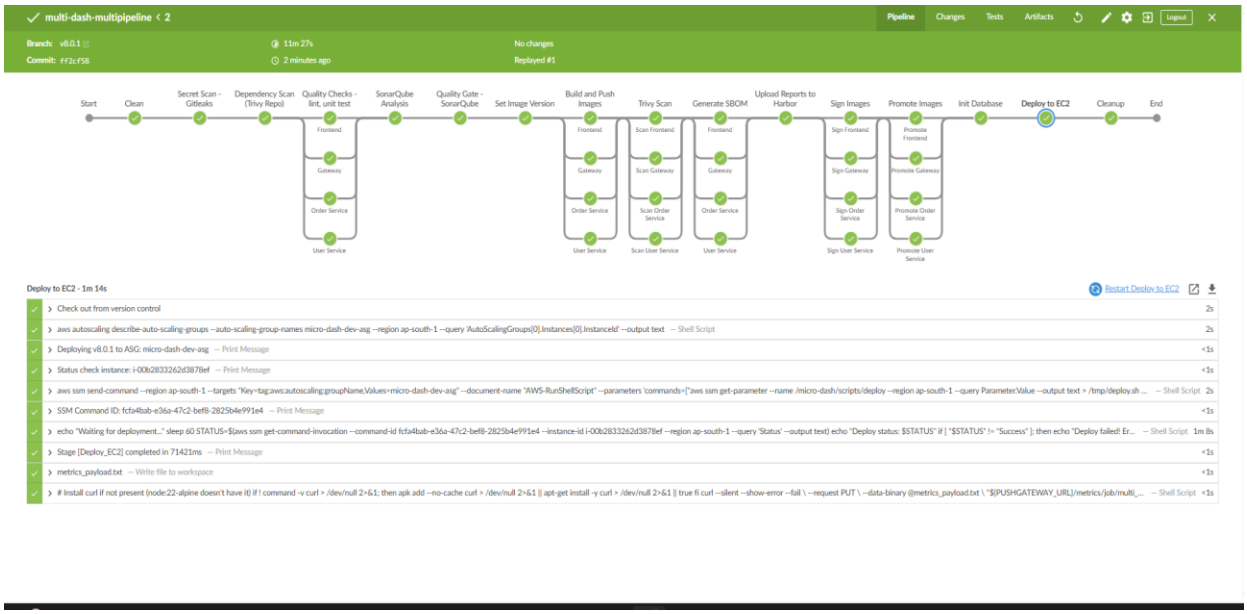

Figure 8: Jenkins Pipeline — all 16 stages green including Deploy to EC2 (branch: v8.0.1, 11m 27s total)

4.1 Pipeline Stages

Stage 1-3: Security & Quality

- Start + Clean — workspace cleanup, environment setup
- Secret Scan (Gitleaks) — scans entire git history for accidentally committed secrets
- Dependency Scan (Trivy Repo) — scans package.json/lock files for vulnerable dependencies

Stage 4-6: Code Quality

- Quality Checks — ESLint linting + Jest unit tests for all 4 services
- SonarQube Analysis — code quality analysis, coverage reports, technical debt
- Quality Gate — fails pipeline if SonarQube quality criteria not met

Stage 7-10: Build & Security Scan

- Set Image Version — derives version from Git tag (e.g., v8.0.1)
- Build and Push Images — Docker BuildKit builds all 4 service images, pushes to Harbor registry
- Trivy Scan — scans built Docker images for OS and application CVEs
- Generate SBOM — creates Software Bill of Materials for each image

Stage 11-13: Registry & Signing

- Upload Reports to Harbor — SBOM and scan reports attached to images in Harbor
- Sign Images — Cosign signs all 4 images with private key for supply chain security
- Promote Images — promotes signed images from Harbor to AWS ECR

Stage 14-16: Deployment

- Init Database — SSM send-command runs db-init.sh on EC2 to create tables and seed data
- Deploy to EC2 — SSM send-command runs deploy.sh on all ASG instances via tag-based targeting
- Cleanup — removes temporary files, sends metrics to Prometheus Pushgateway

4.2 Why SSM Instead of SSH

A key design decision was using AWS Systems Manager (SSM) for deployment instead of traditional SSH. This approach is more secure and practical given that Jenkins runs on GCP while EC2 instances are in AWS private subnets.

- No SSH required — EC2 instances have no port 22 open, reducing attack surface
- IAM authenticated — uses AWS credentials instead of SSH keys
- Works across clouds — Jenkins on GCP can reach AWS EC2 without VPN or peering
- Full audit trail — every SSM command is logged in AWS CloudTrail
- ASG-aware — using --targets with ASG tag deploys to all instances automatically

4.3 SSM Parameter Store for Scripts

Deployment scripts are stored in SSM Parameter Store rather than embedded inline in pipeline commands. This solves the escaping nightmare caused by nested Groovy, JSON, Bash, Python, and SQL string contexts.

- /micro-dash/scripts/db-init — bash script to create tables and seed initial data
- /micro-dash/scripts/deploy — bash script to pull Docker images and restart containers
- Jenkins uploads scripts via aws ssm put-parameter before triggering deployment
- EC2 fetches script via aws ssm get-parameter and executes locally
- Zero JSON escaping issues — scripts stored and retrieved as plain text

5. End-to-End Deployment Flow

The complete deployment flow from code push to live application involves 6 major steps, automated entirely by the Jenkins pipeline triggered by a Git tag push.

Step 1: Developer Pushes Git Tag

- Developer runs: git tag v8.0.1 && git push origin v8.0.1
- Jenkins detects new tag via webhook
- Pipeline starts — Kubernetes agent pod spins up on Minikube

Step 2: CI Stages (Stages 1-13)

- Security scans, unit tests, code quality checks run in parallel where possible
- Docker images built with BuildKit layer caching for speed
- Images tagged with Git tag version (v8.0.1) for traceability
- Images scanned, signed with Cosign, and promoted to ECR

Step 3: Database Initialization (Stage 14)

- Jenkins uploads db-init.sh to SSM Parameter Store
- Jenkins calls aws ssm send-command targeting the ASG
- EC2 instance fetches db-init.sh from Parameter Store
- Script connects to RDS using credentials from Secrets Manager
- Creates tables (IF NOT EXISTS) and seeds initial data (ON CONFLICT DO NOTHING)

Step 4: Application Deployment (Stage 15)

- Jenkins calls aws ssm send-command with --targets ASG tag
- ALL instances in ASG receive the command simultaneously
- Each EC2 fetches deploy.sh and runs it with the image tag as argument
- deploy.sh: ECR login → docker pull → stop old containers → start new containers
- Health checks: curl /health and curl frontend after startup

Step 5: ALB Health Checks

- ALB continuously checks /health endpoint on all registered targets
- New containers start → health check passes → ALB routes traffic
- If any container is unhealthy → ALB stops routing to that instance
- Zero-downtime rolling available when deploying to instances one at a time

Step 6: Verification

- Browser accesses ALB DNS → frontend loads → shows live data
- Dashboard polls /api/dashboard every 5 seconds
- Data flows: ALB → gateway → user-service/order-service → RDS

---

6. Challenges Faced & Solutions

Building this pipeline involved solving multiple real-world DevOps challenges. Each issue encountered reflects common production problems with their practical solutions.

Challenge 1: Cross-Cloud Deployment

- Problem: Jenkins on GCP cannot SSH into EC2 instances in AWS private subnets
- Solution: AWS SSM send-command via AWS API — no network path needed, IAM authenticated

Challenge 2: JSON Escaping in Pipeline

- Problem: Groovy → JSON → Bash → SQL = 4 levels of escaping caused syntax errors
- Solution: Store scripts in SSM Parameter Store — EC2 fetches and runs as plain bash

Challenge 3: Secrets Manager State Loss

- Problem: Terraform destroy marks secrets for 7-day deletion; re-apply fails with ResourceExistsException
- Solution: Added recovery_window_in_days=0 to Terraform for immediate deletion on destroy

Challenge 4: RDS Password Mismatch

- Problem: Terraform random_password generates new password on re-apply but old secret restored
- Solution: Manually reset RDS password and update Secrets Manager to match
- Permanent fix: aws_secretsmanager_secret_version syncs Terraform random password automatically

Challenge 5: SSL Certificate Rejection

- Problem: Node.js rejects RDS SSL certificate: self-signed certificate in certificate chain
- Solution: Set NODE_TLS_REJECT_UNAUTHORIZED=0 environment variable on containers

Challenge 6: DB Schema Mismatch

- Problem: db-init.sh created orders table with product/amount columns but app.js uses item/quantity
- Solution: Recreated orders table with correct schema matching application code

Challenge 7: Instance ID Parsing

- Problem: Multiple EC2 instance IDs concatenated without spaces when using --instance-ids
- Solution: Switched to --targets with ASG tag — SSM finds all instances automatically

7. Security Implementation

7.1 Secrets Management

- All database credentials stored in AWS Secrets Manager — never in environment variables or code
- Application fetches credentials at runtime using EC2 IAM role — no keys stored on EC2
- Separate secrets for user-service and order-service — breach of one does not expose the other

7.2 Network Security

- EC2 instances in private subnets — no direct public internet access
- RDS in private subnets — only accessible from EC2 security group
- ALB in public subnets — only entry point for user traffic
- Security groups follow least privilege — only required ports open between components

7.3 Image Security

- Gitleaks — scans git history for committed secrets before every build
- Trivy — scans both dependency files and built Docker images for CVEs
- Cosign — signs every image with private key; signature stored in Harbor registry
- Distroless base images — minimal OS footprint, no shell or package manager
- SBOM generated — complete inventory of all software components in each image

7.4 Access Control

- Jenkins IAM user: minimal permissions — only ECR, SSM, EC2 describe, autoscaling describe
- EC2 IAM role: ECR pull, Secrets Manager read, SSM managed instance core
- No SSH keys — SSM Session Manager used for all remote access
- CloudTrail enabled — every AWS API call logged for audit

---

8. EC2 Limitations & Next Steps (ECS/EKS)

8.1 EC2 Approach Limitations

- Container crash recovery — Docker --restart flag helps but not as reliable as orchestration
- New ASG instances — user_data only bootstraps OS; containers must be deployed separately via SSM
- Manual rolling deployment — stopping old containers before starting new ones requires scripting
- Resource inefficiency — one EC2 dedicated to one set of containers
- No native container health monitoring — ALB checks EC2 instance, not individual containers

8.2 ECS Fargate Solution

- Managed container orchestration — AWS handles container placement and scheduling
- Task definition versioning — each deployment creates a new task definition version
- Rolling deployment built-in — ECS replaces tasks one by one with health check verification
- Fargate serverless — no EC2 management required; pay per vCPU/memory per second
- ALB target group integration — ALB routes directly to containers, not EC2 instances

8.3 EKS Roadmap

- Helm charts — package all Kubernetes manifests with environment-specific values
- ArgoCD GitOps — declarative deployment; Git is the single source of truth
- Argo Rollouts — blue/green and canary deployment strategies with automatic rollback
- HPA — Horizontal Pod Autoscaler scales pods based on CPU/memory metrics
- Prometheus + Grafana — metrics collection and visualization
- Loki — centralized log aggregation

9. Infrastructure Component Summary

Component	Configuration	Purpose
VPC	10.0.0.0/16, 2 AZs	Network isolation
Public Subnets	10.0.1.0/24, 10.0.2.0/24	ALB and NAT GWs
Private Subnets	10.0.3.0/24, 10.0.4.0/24	EC2 and RDS
NAT Gateway x2	One per AZ, Elastic IP	Outbound internet for private subnets
ALB	Internet-facing, path routing	Traffic distribution
ASG	Desired: 2, Min: 2, Max: 4	Auto scaling and self-healing
EC2	t3.medium, Ubuntu 22.04	Application container host
RDS user-db	PostgreSQL 16.6, Multi-AZ	User data persistence
RDS order-db	PostgreSQL 16.6, Multi-AZ	Order data persistence
Secrets Manager	2 secrets (user + order)	Secure credential storage
ECR	4 repositories	Docker image registry
SSM Parameter Store	2 scripts (db-init, deploy)	Deployment script storage
IAM	EC2 role + Jenkins user	Access control

micro-dash EC2 Deployment — Rajesh | March 2026